Lecture IV - Modelling with JuMP

Applied Optimization with Julia

Dr. Tobias Vlćek



Recap of Last Week

Goals for Today

After this lecture, you will:

- Have **refreshed** functions, packages, DataFrames, IO, and plots
- Know the difference between a **model and a solver**
- Have seen the transport problem from the first lecture **solved in JuMP**
- Be ready to build your first own models in the tutorials

Functions

- Functions are reusable blocks of code
- Define functions using the `function` keyword
- Functions can take arguments and return values
- Use `return` to specify the output of a function
- **This week**: functions help you to structure your models

> **Tip**
> You can create anonymous functions using the `->` syntax for quick, one-off operations.

Packages

- Packages extend Julia's functionality
- Use `using Pkg` to access package management
- Install packages with `Pkg.add("PackageName")`
- Or press `]` in the REPL to enter package mode and type `add PackageName`
- Import packages with `using PackageName` or `import PackageName`
- **This week**: you will install two new packages, `JuMP` and `HiGHS`

DataFrames

- DataFrames are used for working with tabular data
- Create DataFrames using the `DataFrame` constructor
- Access columns using dot notation or square brackets
- Perform operations on columns and rows
- **This week**: DataFrames can hold the data of your models

> **Tip**
> Use `describe(df)` to get a quick summary of your DataFrame.

Input/Output (IO)

- IO operations allow reading from and writing to files
- Reading and writing CSV files can be done with the `CSV` package
- Use `CSV.read("file.csv", DataFrame)` to read a CSV file into a DataFrame
- Use `CSV.write()` to write a DataFrame to a CSV file
- **This week**: this is how you load model data from files

> **Note**
> The `DataFrame` is a **required** second argument of `CSV.read()` — it tells the function into which structure to read the file.

Plots

- Plotting in Julia is done through packages like Plots.jl
- Create basic plots with functions like `plot()`, `scatter()`, `bar()`
- Customize plots with attributes like `title`, `xlabel`, `ylabel`
- **This week**: plots help you to inspect data and results

> **Tip**
> Explore different plot types; later, different backends offer other output formats and interactivity.

Modelling with JuMP

What is JuMP?

Question: How do we get a model from paper into the computer?

- **JuMP** is a package for modelling optimization problems
- We write variables, objective, and constraints **almost like the math**
- JuMP hands the model to a **solver** to solve it
- Ours is **HiGHS**, a free and open-source solver

> **Note**
> Model and solver are separate: JuMP describes **what** to optimize, HiGHS figures out **how** to solve it.

Remember the Solar Panels?

In the first lecture, we formulated the transport problem:

$$
\begin{aligned}
\text{Minimize} \quad & F = \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{J}} c_{i,j} \times X_{i,j} \\
\text{subject to:} \quad & \sum_{j \in \mathcal{J}} X_{i,j} \leq a_i && \forall i \in \mathcal{I} \\
& \sum_{i \in \mathcal{I}} X_{i,j} = b_j && \forall j \in \mathcal{J} \\
& X_{i,j} \geq 0 && \forall i \in \mathcal{I}, \forall j \in \mathcal{J}
\end{aligned}
$$

Let's now solve it — **live in JuMP**!

Step 1: Data, Model, and Variables

```
using JuMP, HiGHS
c = [5010 4640 1980; 7120 1710 6430] # costs per truckload
a = [34, 41]                         # available supply
b = [21, 17, 29]                     # customer demand
transport = Model(HiGHS.Optimizer)
set_silent(transport)                # hide the solver log
@variable(transport, X[1:2, 1:3] >= 0);
```

- First the data, then the model with its solver, then the variables $X_{i,j} \geq 0$

Step 2: Objective and Constraints

```
@objective(transport, Min,
    sum(c[i,j] * X[i,j] for i in 1:2, j in 1:3))
@constraint(transport, [i in 1:2],
    sum(X[i,j] for j in 1:3) <= a[i])
@constraint(transport, [j in 1:3],
    sum(X[i,j] for i in 1:2) == b[j]);
```

- Compare this with the mathematical model: each line of math becomes **one block of code**

Step 3: Solve the Model

```
optimize!(transport)
println("Minimal costs: ", objective_value(transport), " Euro")
```

```
Minimal costs: 225460.0 Euro
```

- Exactly the optimal solution from the first lecture!

> **Note**
> Don't worry if not every line is clear yet — this week's tutorials build up this pattern step by step.

Your Turn

Play: https://beyondsimulations.github.io/Applied-Optimization/games/boards.html

Five Tutorials

Topics of the Tutorials

- **JuMP**: Learn how to use JuMP to define optimization problems
- **Variable Bounds**: Learn how to set variable bounds
- **Constraints**: Learn how to add constraints to your model
- **Advanced Modeling**: Learn how to model more complex problems
- **Transport Problem**: Learn how to solve the transport problem from the first lecture

Get Started with the Tutorials

- Download this week's tutorials and start with the first one
- **Remember, you can ask questions anytime in class or via email!**

> **And that's it for this lecture!**
> The remaining time we will already start working on this week's tutorials. From next week on, we will work on different optimization problems and topics that we will address together in the course.

Literature

- Lauwens, B., & Downey, A. B. (2019). Think Julia: How to think like a computer scientist (First edition). O'Reilly®. Link to the free book website.
- Kwon, C. (2019). Julia Programming for Operations Research (Second edition). Link to the book website.
- Julia Documentation
- JuMP Documentation

For more interesting literature to learn more about Julia, take a look at the literature list of this course.